

ECE 4502/6502 & CS 6501: Machine Learning on Graphs

Lecture 14. Graph Neural Networks III

Instructor: Jundong Li
Spring 2025, University of Virginia

Administrivia

- Homework 2 deadline: March 26th
- Project Progress Report deadline: March 26th, 2 pages in ACM conference format

Recap

- GCN
 - Model training
 - Unsupervised training: no labels available, guided by graph structure
 - Supervised learning: minimize loss between label $\mathbf{y}$ and model prediction $f_{\Theta}(\mathbf{x})$ (L2 norm or cross entropy)
 - Model design overview
 - Define neighborhood aggregation function
 - Define loss function on the embeddings
 - Train on a set of nodes
 - Generate embeddings for nodes as needed

Recap

- GraphSAGE

- Generalize aggregation function & concatenate aggregated embedding and self embedding

- Mean aggregation $\text{AGG} = \sum_{u \in N(v)} \frac{\mathbf{h}_u^{k-1}}{|N(v)|}$
- Mean/Max pool $\text{AGG} = \text{Mean/Max}(\{\text{MLP}(h_u^{(l-1)}), \forall u \in N(v)\})$
- LSTM $\text{AGG} = \text{LSTM}([\mathbf{h}_u^{k-1}, \forall u \in \pi(N(v))])$

- Graph Attention Networks

- Aggregate neighborhood with attention weights

$$\mathbf{h}_v^{(l)} = \sigma(\sum_{u \in N(v)} \alpha_{vu} \mathbf{W}^{(l)} \mathbf{h}_u^{(l-1)})$$

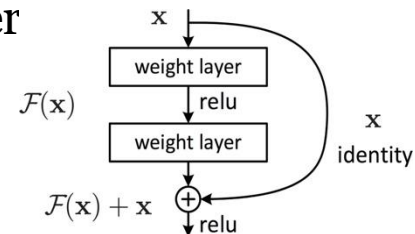
Attention weights

Attention coefficient: $e_{vu} = a(\mathbf{W}^{(l)} \mathbf{h}_u^{(l-1)}, \mathbf{W}^{(l)} \mathbf{h}_v^{(l-1)})$

Attention weight: $\alpha_{vu} = \frac{\exp(e_{vu})}{\sum_{k \in N(v)} \exp(e_{vk})}$

Recap

- GNN layers in practice
 - Batch normalization
 - Dropout
 - Activation (ReLU, Sigmoid, Parametric ReLU)
 - Attention
- Stacking GNN layers
 - Over-smoothing problem
 - Increase expressive power within GNN layers
 - Use MLP as aggregation/transformation functions
 - Pre-processing/post-processing layer
 - Add skip connections in GNN



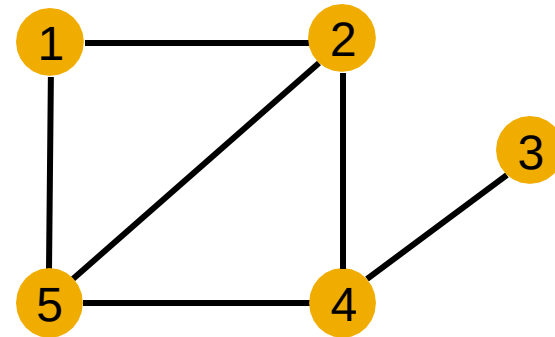
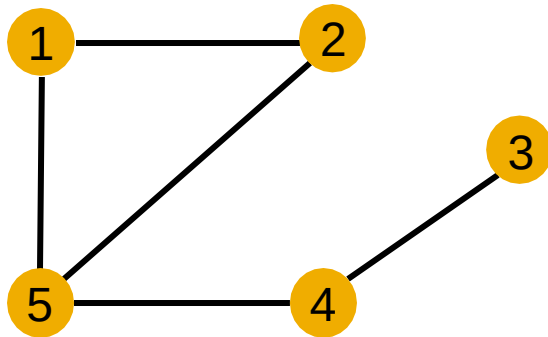
Theory of Graph Neural Networks

Theory of GNNs

- How powerful are GNNs?
 - Many GNN models have been proposed (e.g., GCN, GAT, GraphSAGE, design space).
 - What is the expressive power (**ability to distinguish different graph structures**) of these GNN models?
 - How to design a maximally expressive GNN model?

Graph Isomorphism Problem

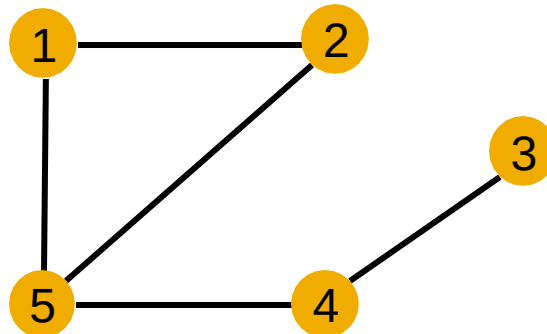
- Graph Isomorphism problem
 - Given a pair of graphs, how to tell if they are isomorphic?



- Open problem (a polynomial time algorithm does not exist, has not been proven NP-complete).

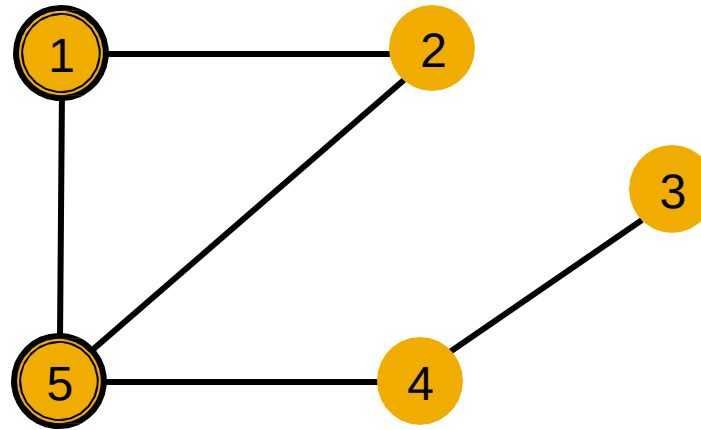
A Simpler Problem

- Can GNN differentiate a pair of nodes with different neighborhood structures?
 - We use node same/different **colors** to represent nodes with same/different features.
 - For example, the graph below assumes all the nodes **share the same feature**.



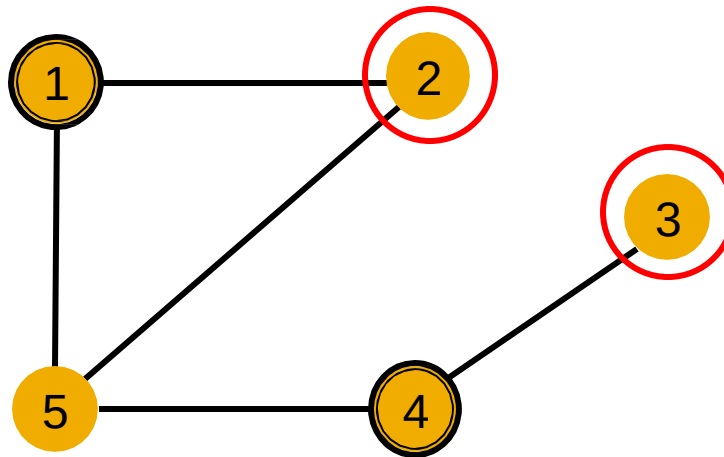
Local Neighborhood Structures

- We specifically consider **local neighborhood structures** around each node in a graph
 - **Example:** Nodes 1 and 5 have **different** neighborhood structures because they have different node degrees.



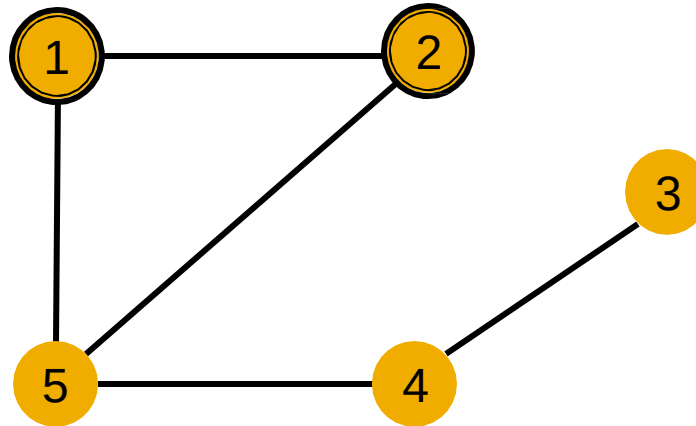
Local Neighborhood Structures

- We specifically consider **local neighborhood structures** around each node in a graph
 - **Example:** Nodes 1 and 4 both have the same node degree of 2. However, they still have **different** neighborhood structures because **their neighbors have different node degrees.**



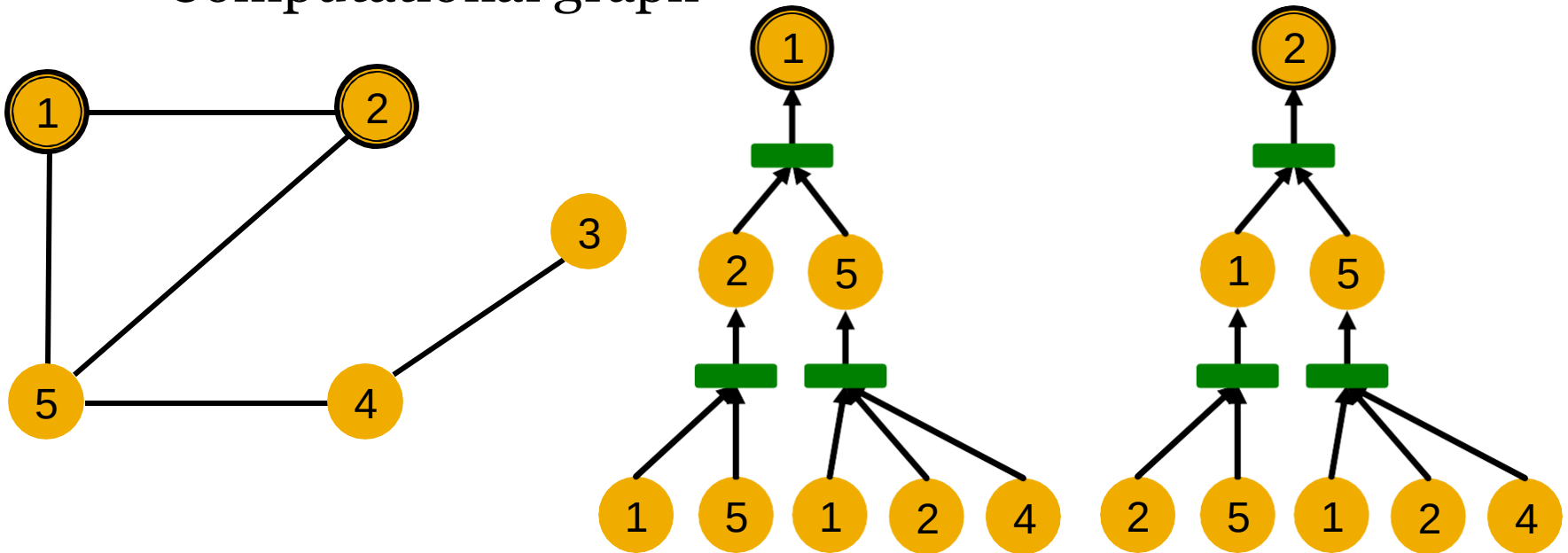
Local Neighborhood Structures

- We specifically consider **local neighborhood structures** around each node in a graph
 - **Example:** Nodes 1 and 2 have the **same** neighborhood structure because **they are symmetric within the graph.**



Local Neighborhood Structures

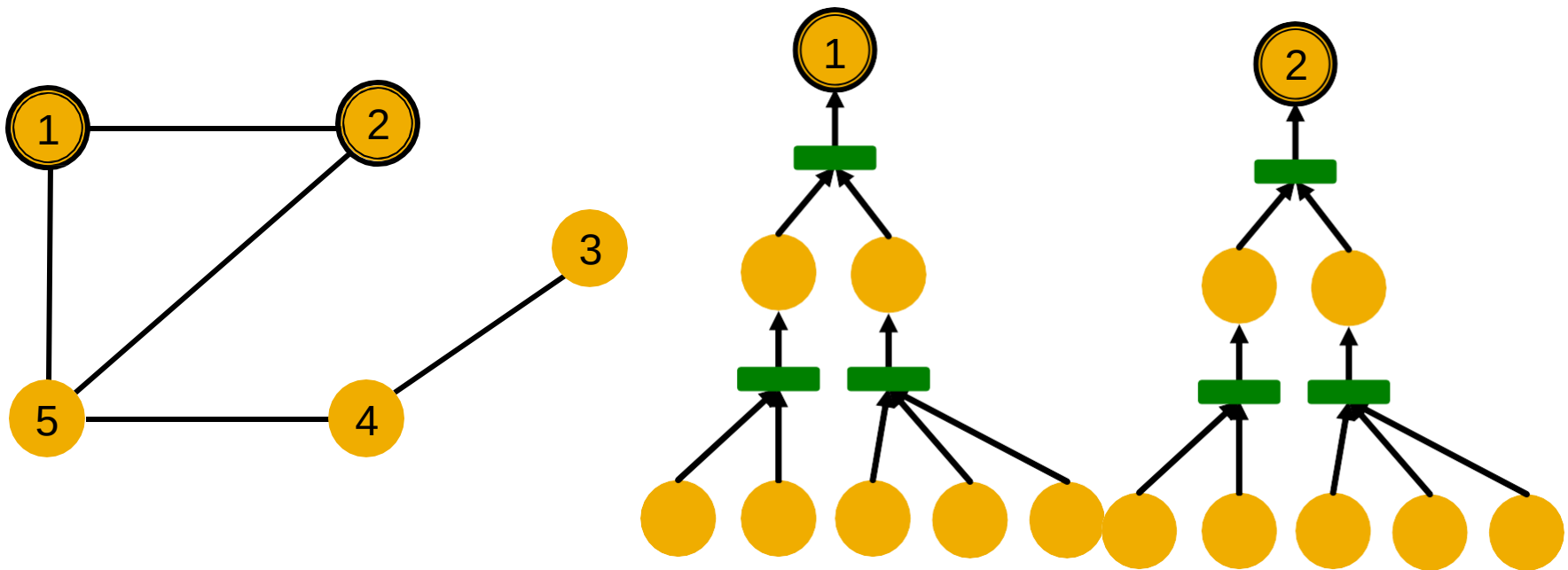
- How a GNN capture local neighborhood structures?
 - Computational graph



Computational Graph

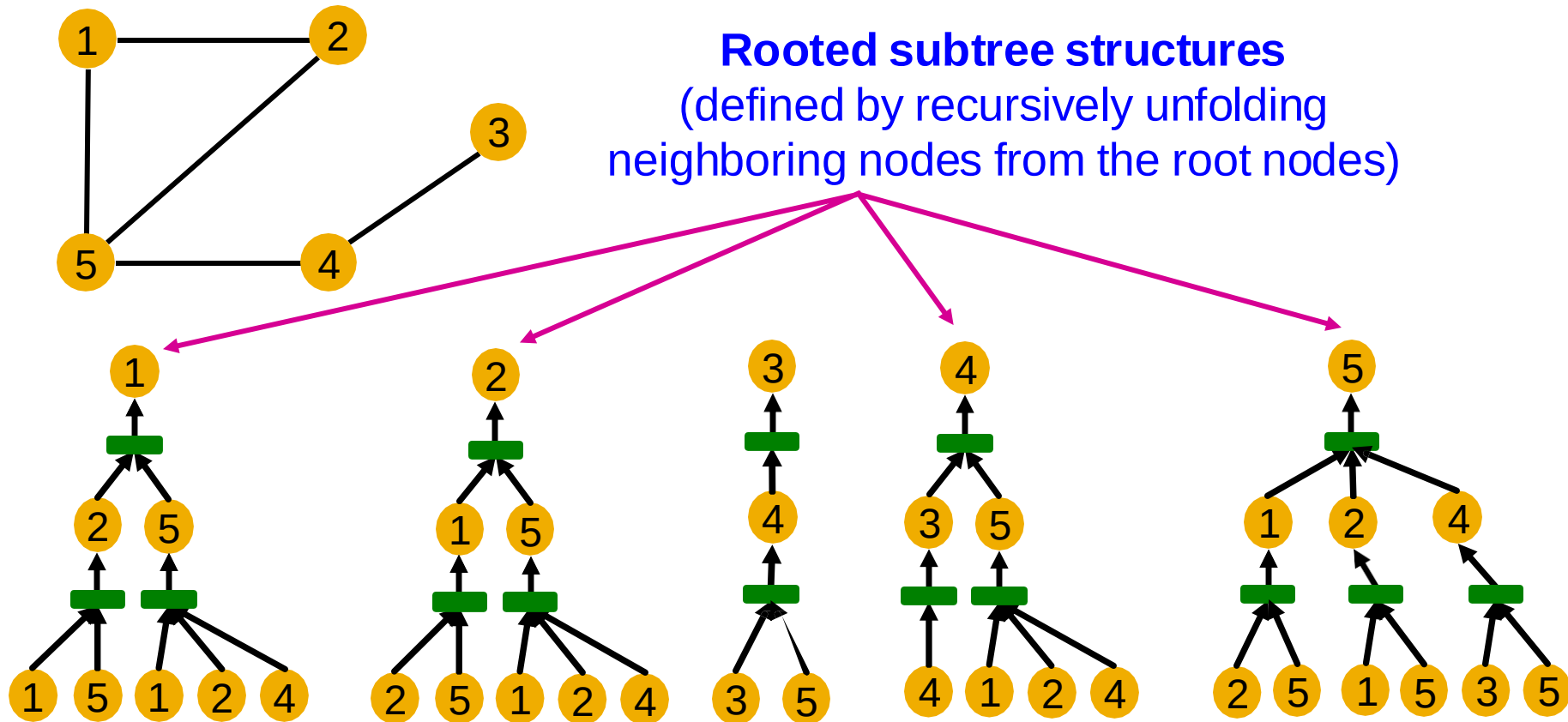
- GNN will generate **the same embedding** for nodes 1 and 2
 - Computational graphs are the same.
 - Node features (colors) are identical.

GNN doesn't care about node ids, it just aggregates features vectors of different nodes.



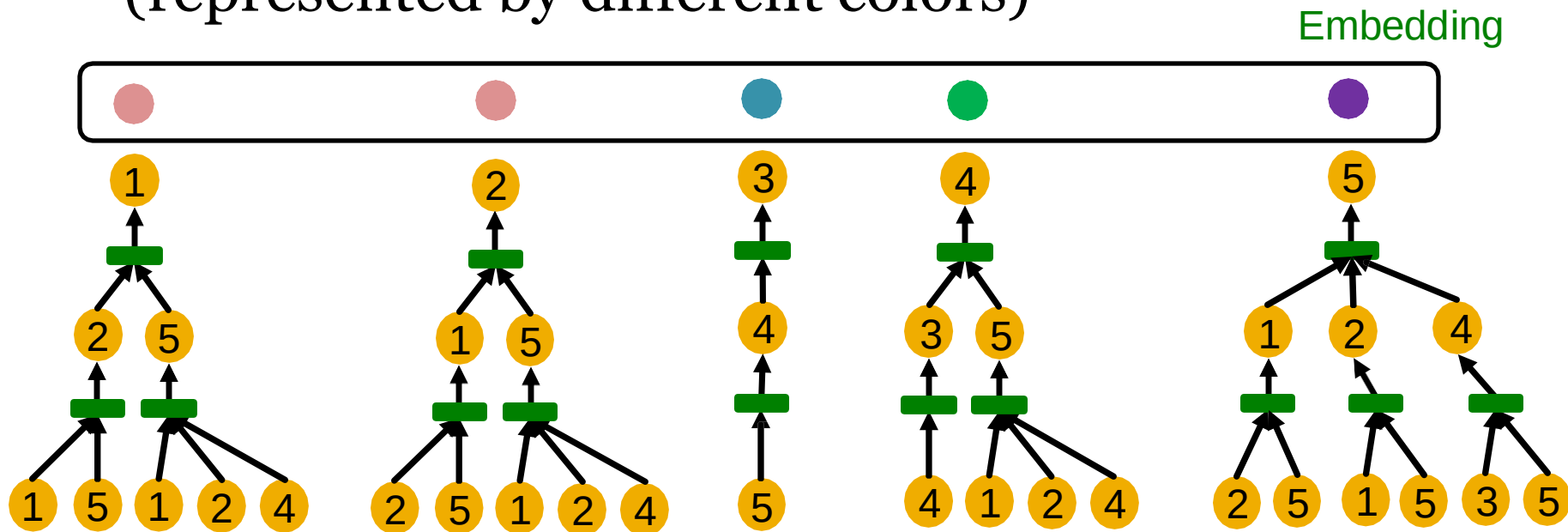
Computational Graph

- Local neighborhood structure defines computational graphs (**rooted subtree structures**)



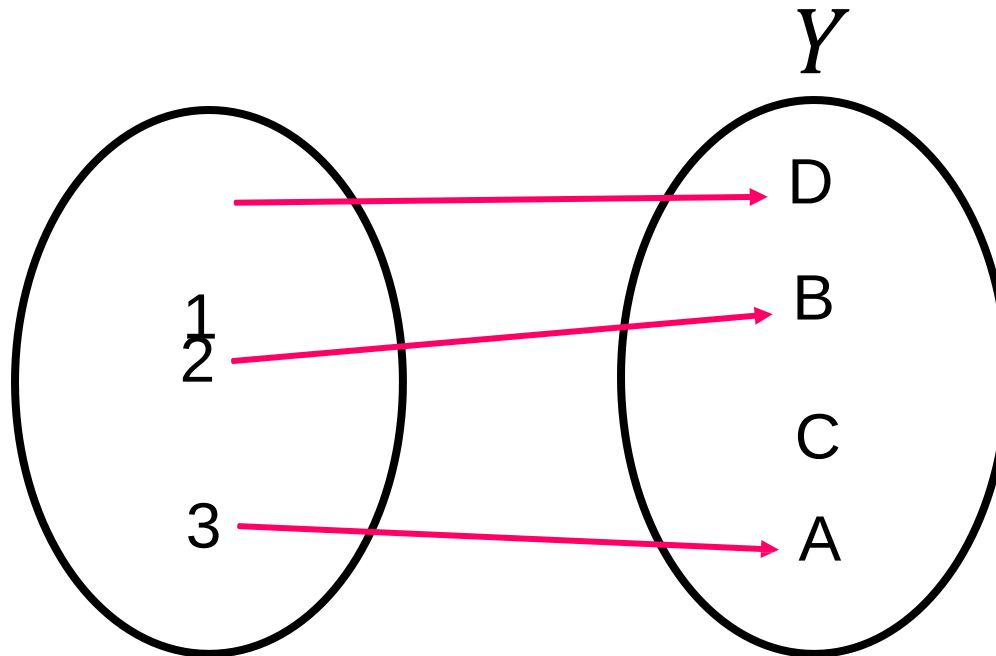
Computational Graph

- GNN's node embeddings capture **rooted subtree structures**.
- Most expressive GNN maps different **rooted subtrees** into different node embeddings (represented by different colors)



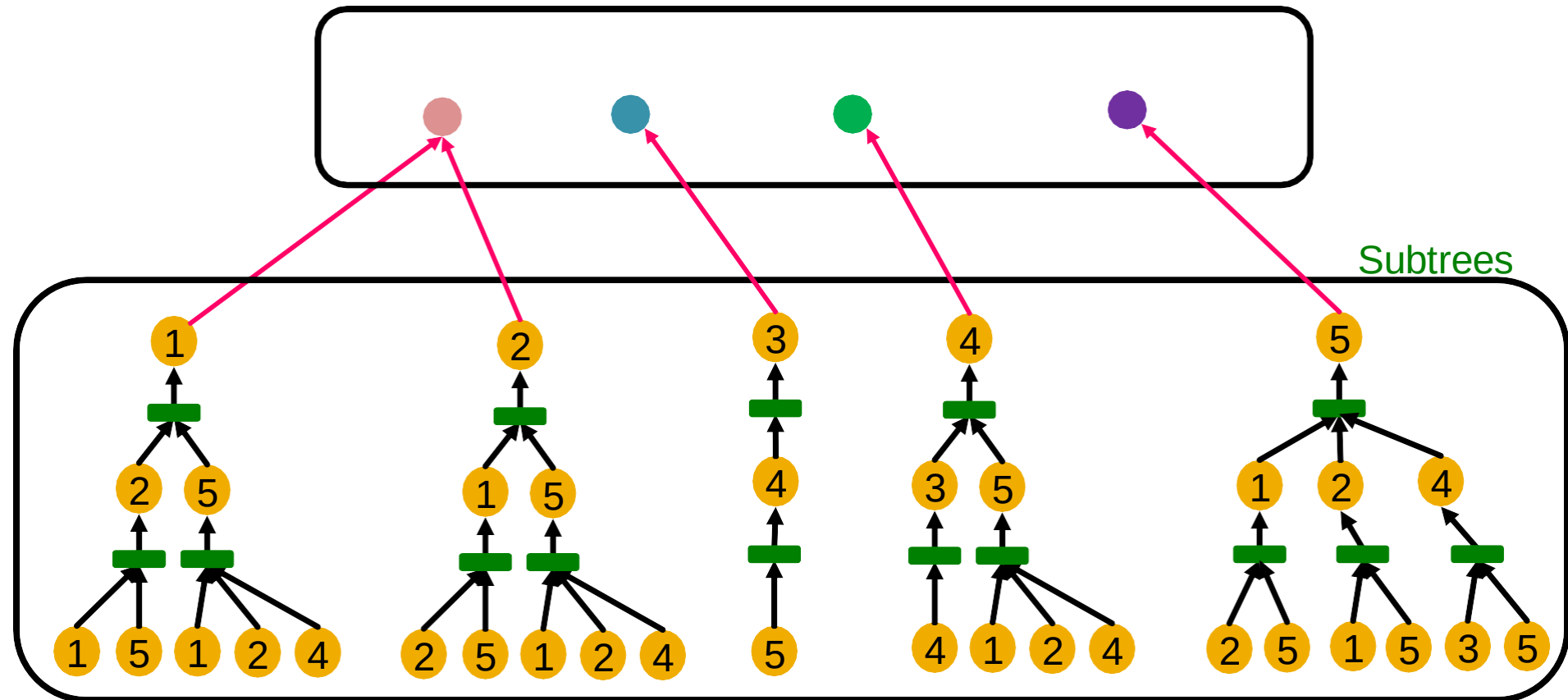
Injective Function

- **Function** $f : X \rightarrow Y$ is **injective** if it maps different elements into different outputs.
 - f retains all the information about input.



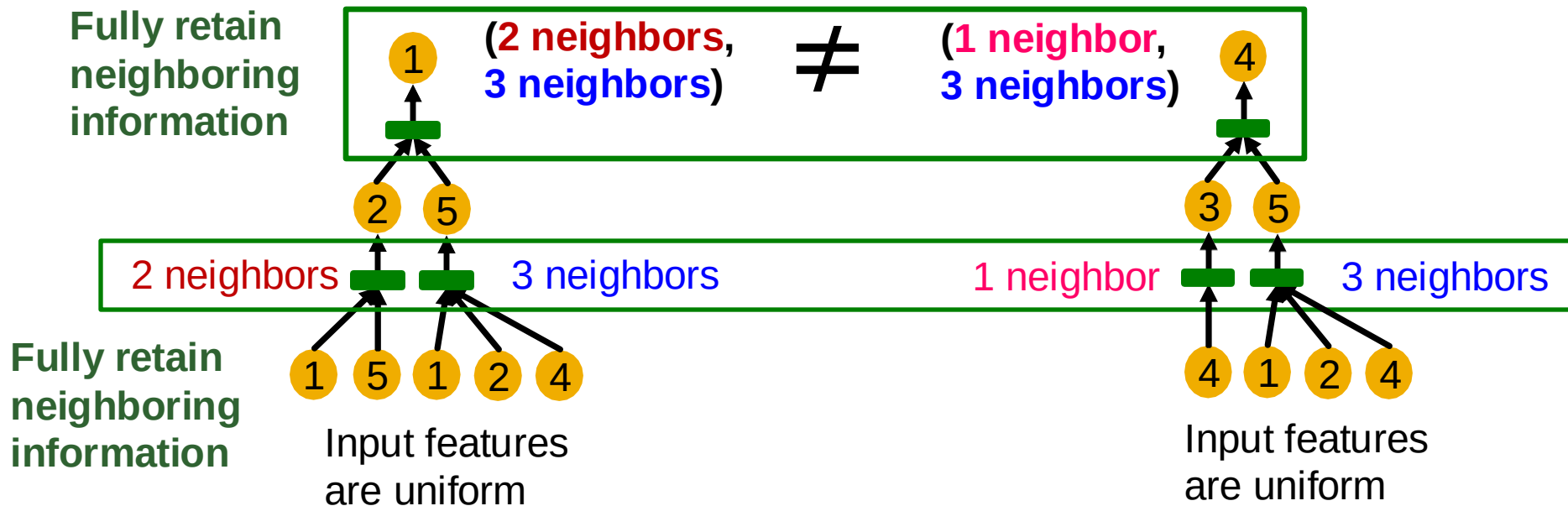
Expressiveness of GNN

- Most expressive GNN should map subtrees to the node embeddings **injectively**.



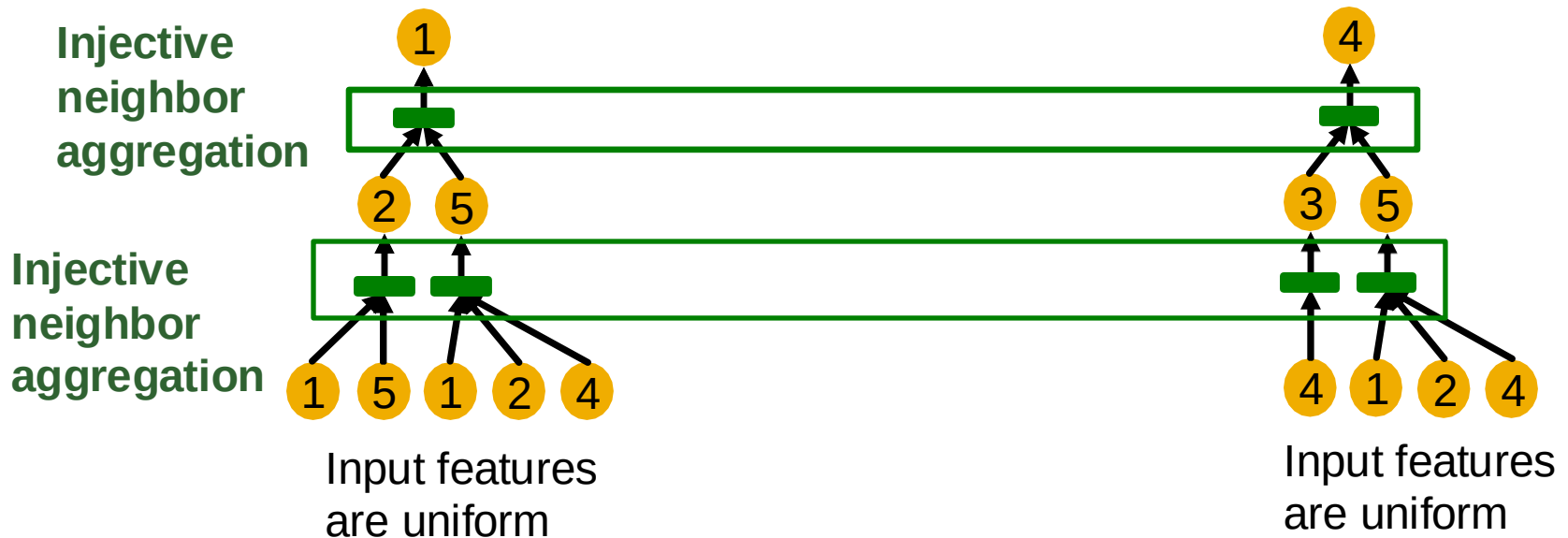
Expressiveness of GNN

- Node embedding is characterized by the **subtree structure** and **aggregation function**
 - If each step of GNN's aggregation **can fully retain the neighboring information**, the generated node embeddings can distinguish different rooted subtrees.



Expressiveness of GNN

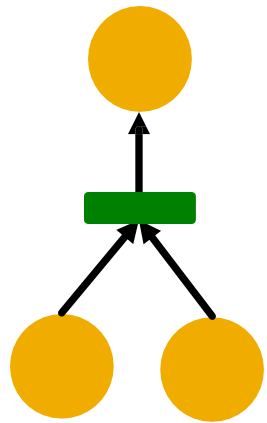
- Most expressive GNN would use an **injective neighbor aggregation** function at each step



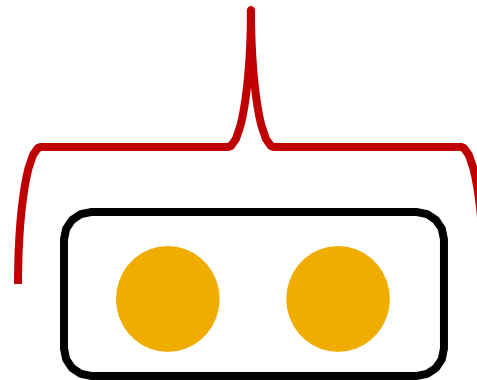
Designing the Most Powerful Graph Neural Network

Neighbor Aggregation

- **Observation:** Neighbor aggregation can be abstracted as a function over a **multi-set** (a set with repeating elements)

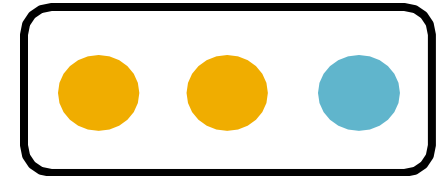
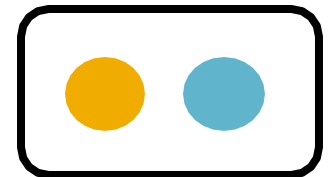


Neighbor
aggregation



Multi-set function

Examples of
multi-set



Same color indicates the
same features.

Revisit Existing GNN models

- **GCN (mean-pool)** [Kipf & Welling, ICLR 2017]
 - Uses **element-wise** mean pooling over neighboring node features

$$\text{Mean}(\{x_u\}_{u \in N(v)})$$

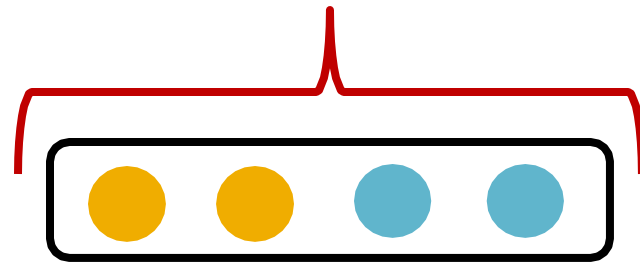
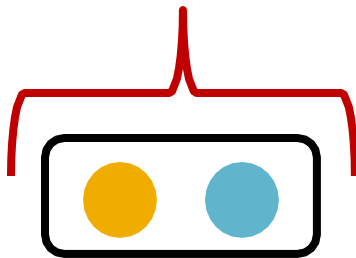
- **GraphSAGE (max-pool)** [Hamilton et al. NeurIPS 2017]
 - Uses **element-wise** max pooling over neighboring node features

$$\text{Max}(\{x_u\}_{u \in N(v)})$$

Neighbor Aggregation: Case Study

- **GCN (mean-pool)** [Kipf & Welling ICLR 2017]
 - Take **element-wise mean**, followed by linear function and ReLU activation, i.e., $\max(0, x)$.
 - **Theorem** [Xu et al. ICLR 2019]
 - GCN cannot distinguish different **multi-sets with the same color proportion**.

Failure Case



- **Why?**

Neighbor Aggregation: Case Study

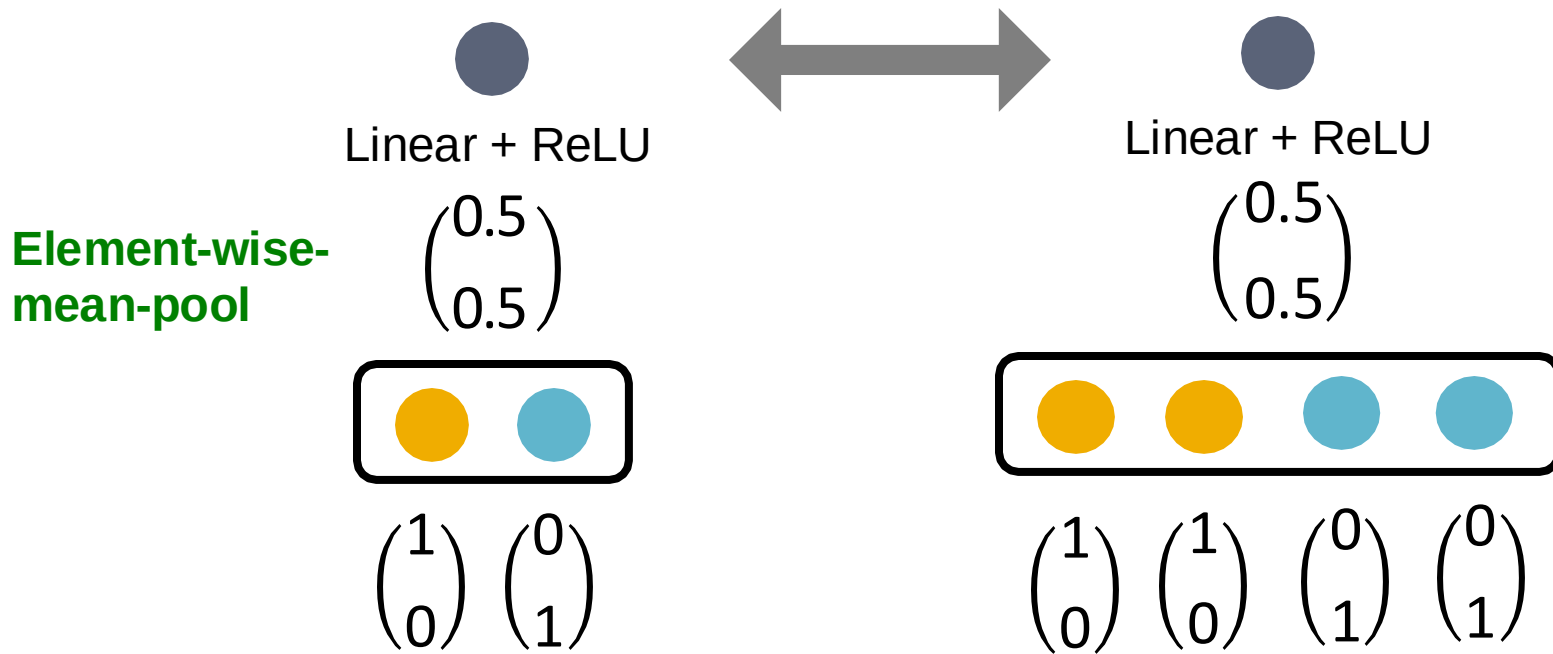
- Assume node features (colors) are represented by **one-hot encoding**
 - **Example:** If there are two distinct colors:

$$\text{Yellow Circle} = \begin{pmatrix} 1 \\ 0 \end{pmatrix} \quad \text{Blue Circle} = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$$

- This assumption is sufficient to illustrate how GCN fails.

Neighbor Aggregation: Case Study

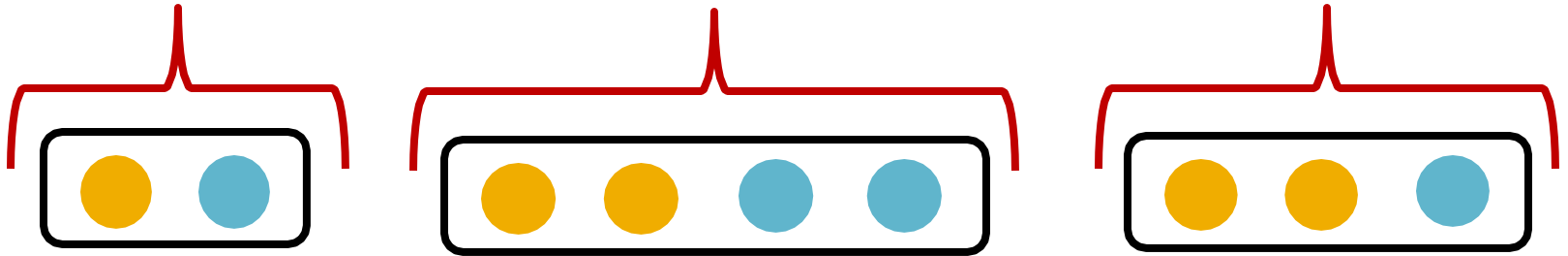
- **GCN (mean-pool)** [Kipf & Welling ICLR 2017]
 - **Failure case illustration**



Neighbor Aggregation: Case Study

- **GraphSAGE (max-pool)** [Hamilton et al. NeurIPS 2017]
 - Apply an MLP, then take **element-wise max**.
 - **Theorem** [Xu et al. ICLR 2019]
 - GraphSAGE cannot distinguish different **multi-sets** with the same set of distinct colors

Failure Case



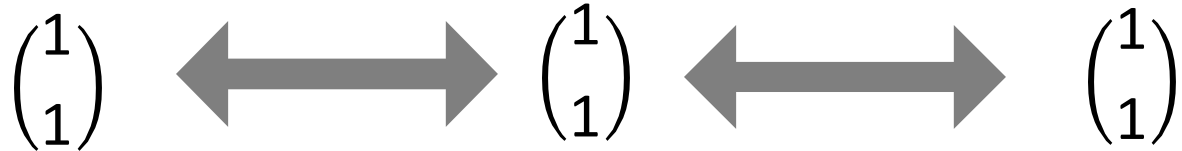
- **Why?**

Neighbor Aggregation: Case Study

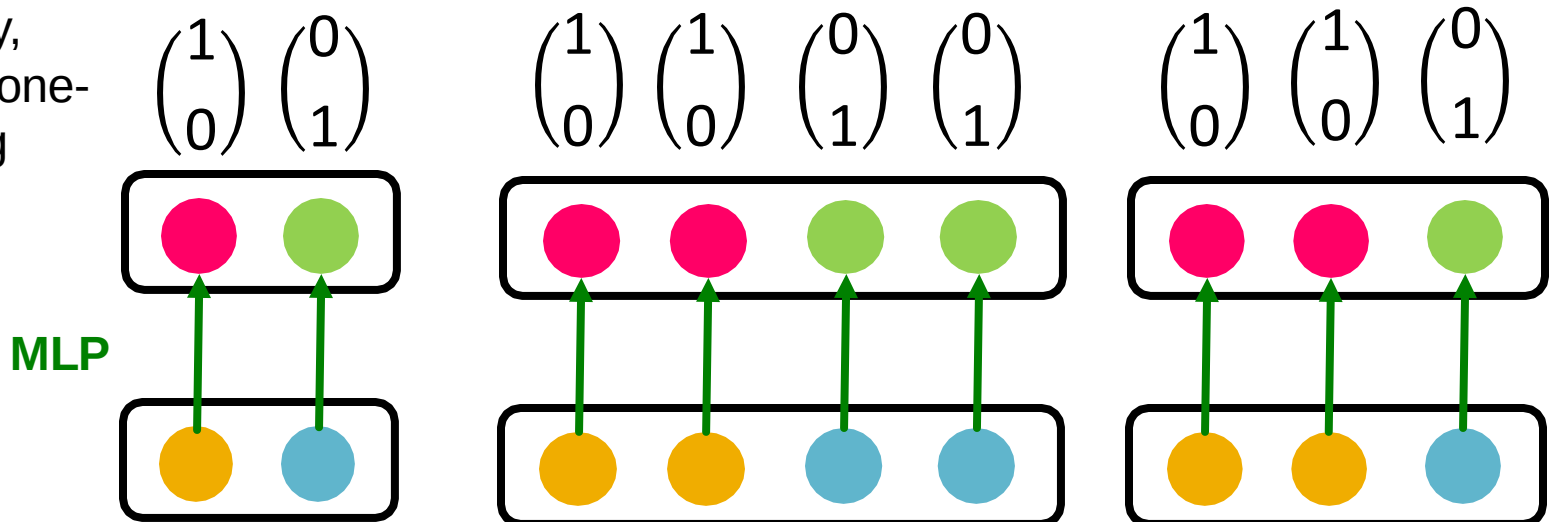
- **GraphSAGE (max-pool)** [Hamilton et al. NeurIPS 2017]
 - **Failure case illustration**

Same Outputs!

Element-wise-max-pool



For simplicity, assume the one-hot encoding after **MLP**.



Summary

- Expressiveness of GNNs
 - Expressive power of GNNs is characterized by the neighbor aggregation function
 - Neighbor aggregation is a function over multi-sets (sets with repeating elements)
 - GCN and GraphSAGE's aggregation functions fail to distinguish some basic multi-sets; hence **not injective**.
 - Therefore, GCN and GraphSAGE are **not** maximally powerful GNNs.

Design Most Expressive GNNs

- Goal
 - Design maximally powerful GNNs in the class of message-passing GNNs.
- ➔ Designing **injective** neighbor aggregation function over multi-sets
- ➔ Use **neural network** as neighbor aggregation function

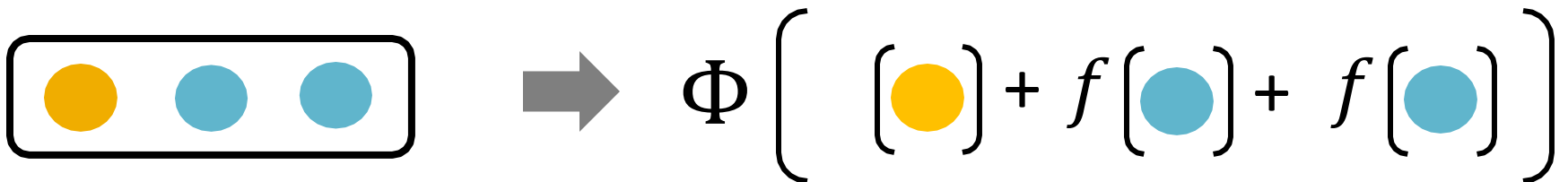
Injective Multi-Set Function

- **Theorem** [Xu et al. ICLR 2019]
 - Any injective multi-set function can be expressed as:

$$\Phi\left(\sum_{x \in S} f(x)\right)$$

Φ, f : Some non-linear functions
 S : a multi-set

S : multi-set



Injective Multi-Set Function

- Proof Intuition

- f produces one-hot encodings of colors. Summation of the one-hot encodings retains all the information about the input multi-set.

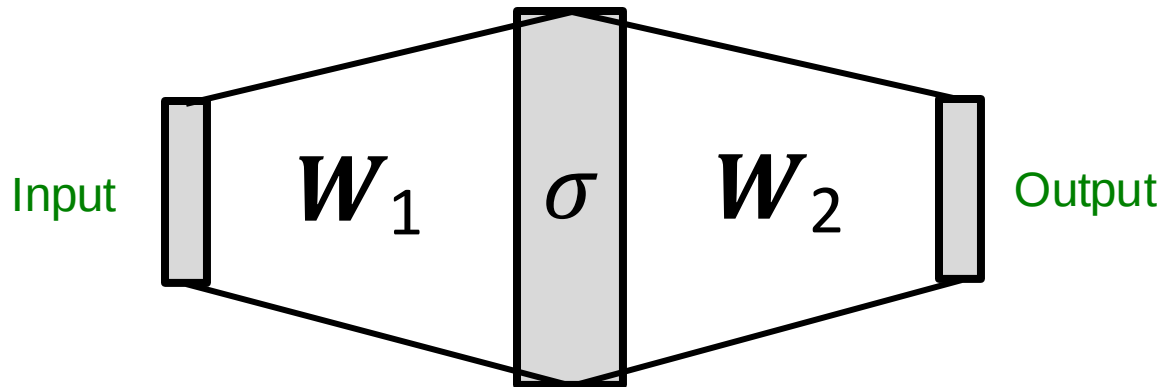
$$\Phi\left(\sum_{x \in S} f(x)\right)$$

Example: $\Phi\left[\underbrace{\left[\text{Yellow Circle}\right]}_{\text{One-hot } \begin{pmatrix} 1 \\ 0 \end{pmatrix}} + \underbrace{f\left[\text{Blue Circle}\right]}_{\begin{pmatrix} 0 \\ 1 \end{pmatrix}} + \underbrace{f\left[\text{Blue Circle}\right]}_{\begin{pmatrix} 0 \\ 1 \end{pmatrix}}\right]$

$$\begin{pmatrix} 1 \\ 0 \end{pmatrix} + \begin{pmatrix} 0 \\ 1 \end{pmatrix} + \begin{pmatrix} 0 \\ 1 \end{pmatrix} = \begin{pmatrix} 1 \\ 2 \end{pmatrix}$$

Universal Approximation Theorem

- How to model Φ and f in the aggregation function?
 - Mult-Layer Perceptron (MLP)
 - **Universal Approximation Theorem** [Hornik et al., 1989]
 - 1-hidden-layer MLP with sufficiently-large hidden dimensionality and appropriate non-linearity $\sigma(\cdot)$ (including ReLU and sigmoid) can **approximate any continuous function to an arbitrary accuracy.**



Injective Multi-Set Function

- We can use a **neural network** to model any injective multi-set function

$$\Phi\left(\sum_{x \in S} f(x)\right) \quad \longrightarrow \quad \text{MLP}_{\Phi}\left(\sum_{x \in S} \text{MLP}_f(x)\right)$$

- In practice, MLP hidden dimensionality of 100 to 500 is sufficient

Most Expressive GNN

- Graph Isomorphism Network (GIN) [Xu et al. ICLR 2019]

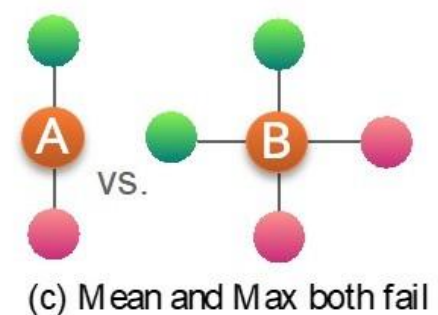
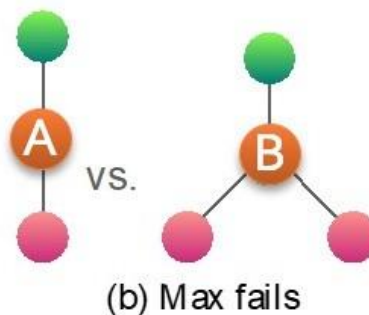
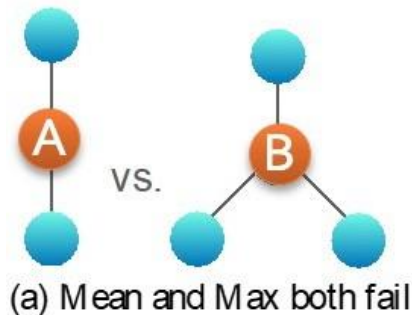
- Apply an MLP, element-wise **sum**, followed by another MLP.

$$\text{MLP}_{\Phi}(\sum_{x \in \mathcal{S}} \text{MLP}_f(x))$$

- Theorem
 - GIN's neighbor aggregation function is injective
 - **No failure cases!**
 - **GIN is the most expressive GNN in the class of message-passing GNNs we have introduced!**

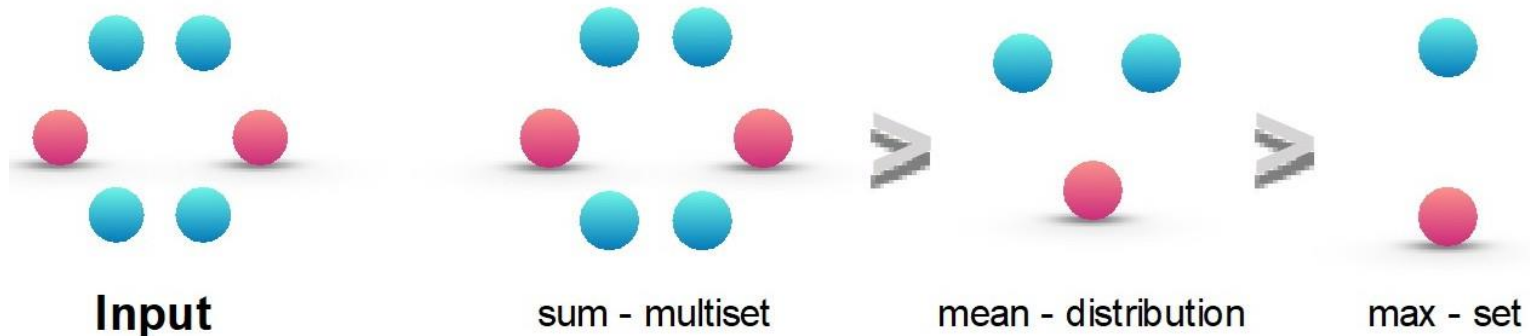
Discussion: The Power of Pooling

- Failure cases for mean and max pooling



Colors represent feature values

- Ranking by discriminative power



Relation to WL Graph Kernel

- GIN is a “neural network” version of the Weisfeiler-Lehman (WL) graph kernel
 - WL graph kernel: compute the similarity of two graphs using the histogram vectors of both graphs
 - A heuristic algorithm for the graph isomorphism problem

WL Graph Kernel

- Color refinement algorithm in WL kernel

- **Given:** A graph G with a set of nodes V

- Assign an initial color $c^{(0)}(v)$ to each node v .
 - Iteratively refine node colors by

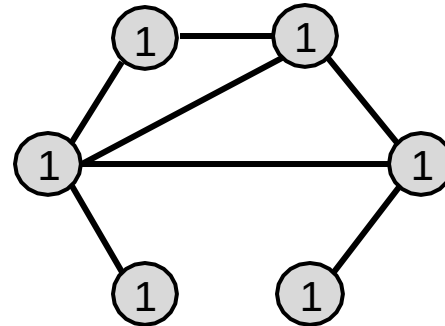
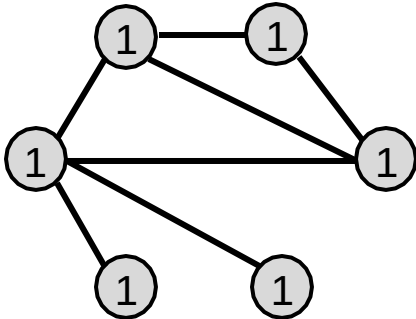
$$c^{(k+1)}(v) = \text{HASH}(c^{(k)}(v), \{c^{(k)}(u)\}_{u \in N(v)})$$

where HASH maps different inputs to different colors

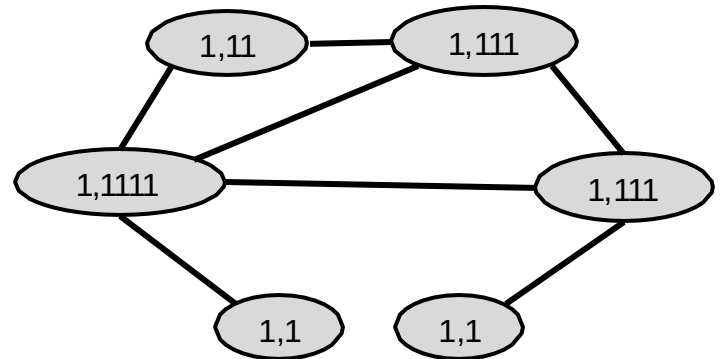
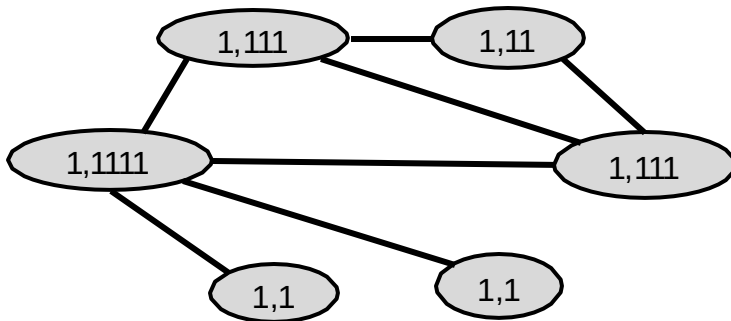
- After K steps of color refinement, $c^{(K)}(v)$ summarizes the structure of **K -hop neighborhood**

Color Refinement

- Example of color refinement given two graphs
 - Assign initial colors

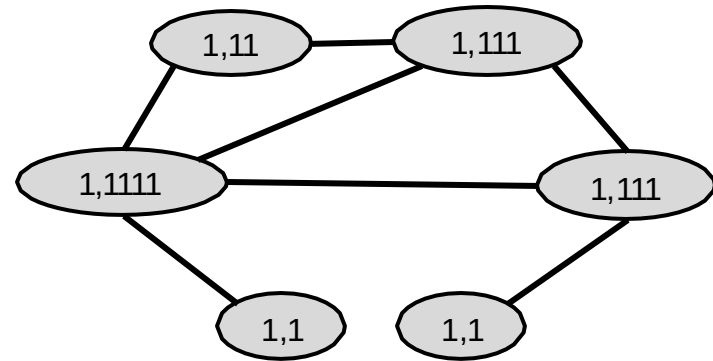
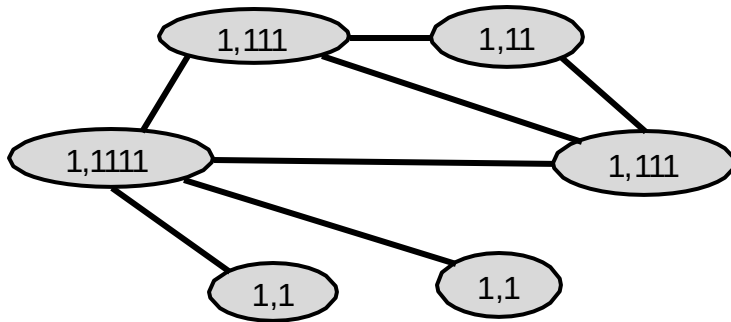


- Aggregate neighboring colors

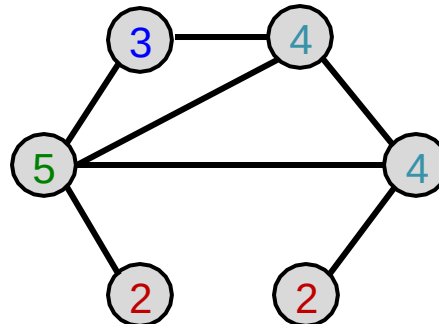
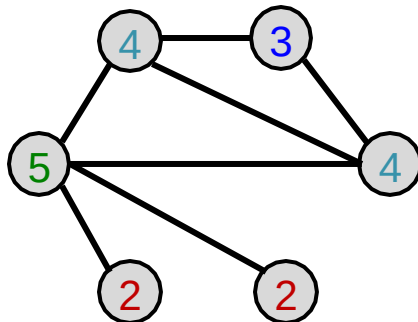


Color Refinement

- Aggregated colors



- Injectively HASH the aggregated colors

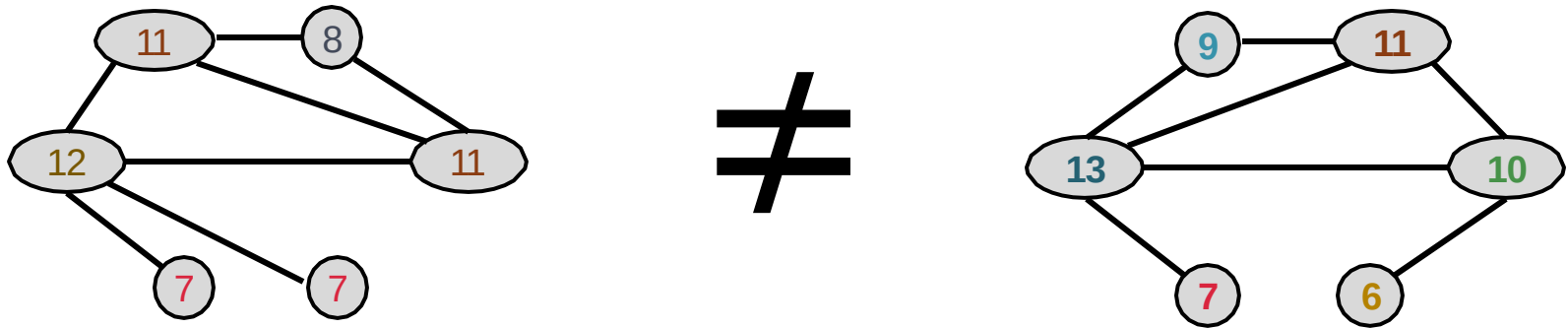


HASH table: **Injective!**

1,1	-->	2
1,11	-->	3
1,111	-->	4
1,1111	-->	5

Color Refinement

- Process continues until a stable coloring is reached
- Two graphs are considered **isomorphic** if they have the same set of colors.



The Complete GIN Model

- GIN uses a **neural network** to model the injective HASH function.

$$c^{(k+1)}(v) = \text{HASH}(c^{(k)}(v), \{c^{(k)}(u)\}_{u \in N(v)})$$

- Specifically, the injective function is applied over the tuple

$$(c^{(k)}(v), \{c^{(k)}(u)\}_{u \in N(v)})$$

Root node
features

Neighboring
node features

The Complete GIN Model

- **Theorem** [Xu et al. ICLR 2019]

- Any injective function over the tuple

$$(c^{(k)}(v), \{c^{(k)}(u)\}_{u \in N(v)})$$

can be modeled as

$$\text{MLP}_{\Phi} \left((1 + \epsilon) \cdot \text{MLP}_f(c^{(k)}(v)) + \sum_{u \in N(v)} \text{MLP}_f(c^{(k)}(u)) \right)$$

where ϵ is a learnable scalar

- **Special case:** if input feature $c^{(0)}(v)$ is one-hot vector, direct summation is injective
 - Only need to ensure the injectivity of Φ

The Complete GIN Model

- Given: a graph G with a set of nodes V
 - Assign an **initial vector** $c^{(0)}(v)$ to each node v
 - Iteratively update node vectors by

$$c^{(k+1)}(v) = \text{GINConv}\left(\left\{c^{(k)}(v), \left\{c^{(k)}(u)\right\}_{u \in N(v)}\right\}\right)$$

Differentiable color HASH function

where **GINConv** maps different inputs to different embeddings

- After K steps of GIN iterations, $c^{(K)}(v)$ summarizes the structure of **K -hop neighborhood**.

GIN vs. WL Graph Kernel

- GIN can be viewed as **differentiable neural version** of the WL graph kernel

	Update target	Update function
WL Graph Kernel	Node colors (one-hot)	HASH
GIN	Node embeddings (low-dim vectors)	GINConv

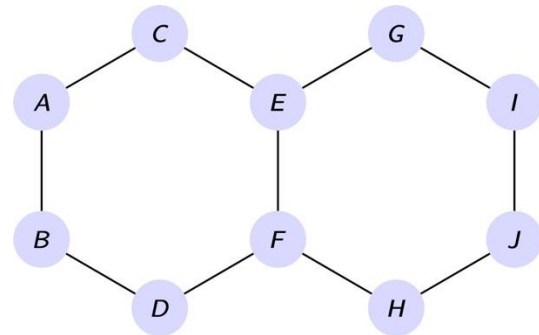
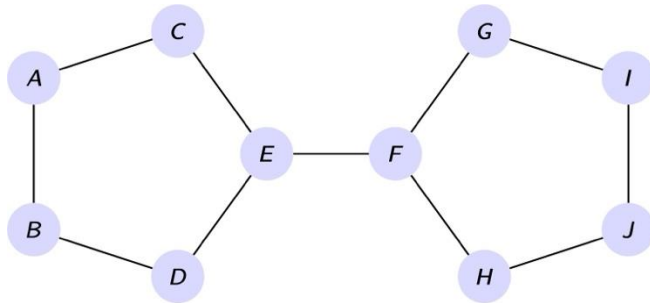
- Advantages of GIN over the WL graph kernel
 - Node embeddings are **low-dimensional**; hence, they can capture the fine-grained similarity of different nodes.
 - Parameters of the update function can be **learned for the downstream tasks**.

Expressive Power of GIN

- The expressiveness of GIN and WL graph kernel is the same
 - If two graphs can be distinguished by GIN, they can be also distinguished by the WL kernel, and vice versa.
- How powerful in practice?
 - WL kernel has been both theoretically and empirically shown to distinguish most of the real-world graphs [Cai et al. 1992].
 - Hence, GIN is also powerful enough to distinguish most of the real graphs!

Some Bad News

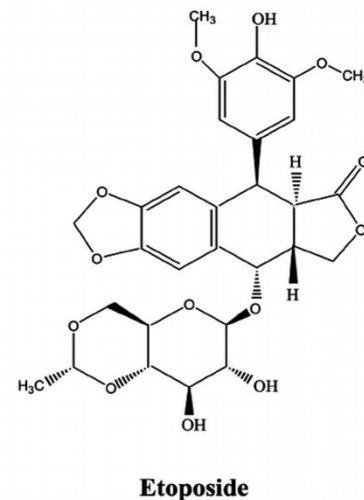
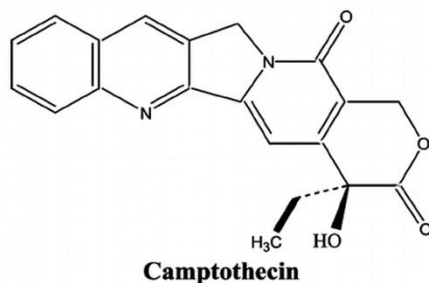
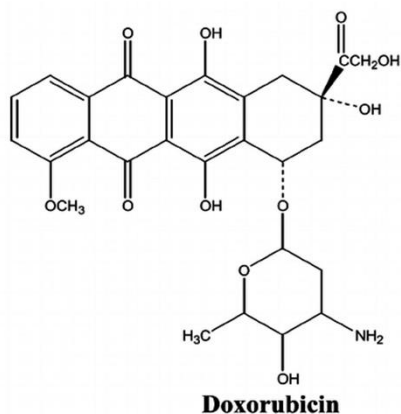
- The WL kernel cannot distinguish between some **basic graph structures**, e.g.,



- The WL kernel cannot count important sub-structures, e.g., cycles and cliques.

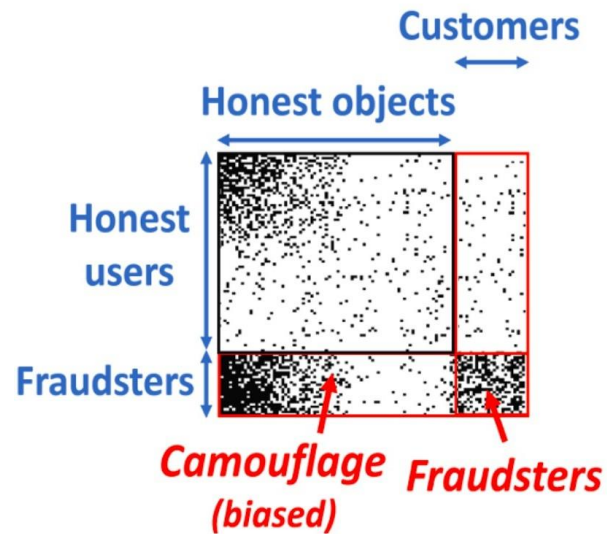
Why Do We Care about Cycles?

- The bonds between different atoms in chemical compounds form **cycles**.
- The **hexagon** is the most common polygon in organic compounds $\Rightarrow$ yields stable materials.



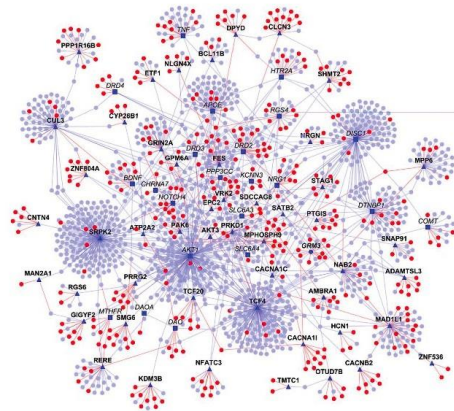
Why Cliques?

Fraudster Detection in e-Commerce Networks



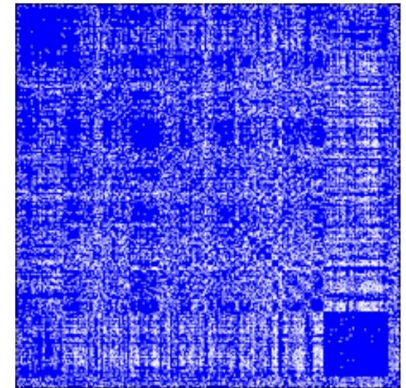
[Hooi et al. 2016]

Detection of Protein Complexes in PPI Networks



[Vlaic et al. 2017]

Graph Compression



Dhulipala et al. 2017

Improving GNN's Power

- The expressiveness of GNN can be further improved to solve the above problems
 - Li et al. NeurIPS 2020
 - You et al. AAAI 2021
 - Kanatsoulis et al. ICLR 2024



Questions?